

1. Least square optimization problem

For an over constrained linear system $\mathbf{Ax}=\mathbf{b}$, find the least square solution of $\mathbf{x}$ using

- Pseudo inverse $\hat{\mathbf{x}} = \mathbf{A}^+ \cdot \mathbf{b}$
- QR decomposition,
- Compare if a) and b) yield the same result?

Note: the pseudo inverse function is `pinv()` in matlab

$$\mathbf{A}_{8 \times 4} = \begin{bmatrix} 15 & -13 & 20 & -8 \\ -5 & -15 & -4 & -4 \\ -17 & 16 & -2 & 9 \\ 10 & -19 & -14 & -15 \\ -7 & 8 & -7 & 15 \\ 14 & 10 & -8 & -17 \\ -5 & -3 & 16 & -2 \\ 13 & -5 & -10 & -19 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 13 \\ 10 \\ -15 \\ 9 \\ 3 \\ 18 \\ 3 \\ 20 \end{bmatrix}$$

(a) pseudo inverse Matlab code:

```
>> A = [15, -13, 20, -8;
        -5, -15, -4, -4;
        -17, 16, -2, 9;
        10, -19, -14, -15;
        -7, 8, -7, 15;
        14, 10, -8, -17;
        -5, -3, 16, -2;
        13, -5, -10, -19];
>> b = [13; 10; -15; 9; 3; 18; 3; 20];
>> x_hat = pinv(A) * b;
>> disp(x_hat);
    0.4638
   -0.1005
   -0.0716
   -0.4137
```

(b) QR decomposition Matlab code:

```
>> [Q, R] = qr(A, 'econ');
>> x_qr = R \ (Q' * b);
>> disp(x_qr);
    0.4638
   -0.1005
   -0.0716
   -0.4137
```

為 Householder 矩陣實作由 Matlab 提供語法

因此改用下面的 Givens Rotation method

Given's Rotation 可硬體化做法：

```
>> [m, n] = size(A);
>> R = A;
>> Q = eye(m);
>> for j = 1:n
    for i = m:-1:j+1
        a_q = R(i-1,j);
        a_q1 = R(i,j);
        if a_q1 ~= 0
            t = a_q1 / a_q;
            cos_theta = 1 / sqrt(1 + t^2);
            sin_theta = t / sqrt(1 + t^2);
            G = [cos_theta, sin_theta; -sin_theta, cos_theta];
            R([i-1, i], j:n) = G * R([i-1, i], j:n);
            G_full = eye(m);
            G_full([i-1, i], [i-1, i]) = G;
            Q = Q * G_full';
        end
    end
end
>> bp = Q' * b;
>> bu = bp(1:n, :);
>> Ru = R(1:n, :);
>> x_gr = Ru \ bu;
>> disp(x_gr);
    0.4638
   -0.1005
   -0.0716
   -0.4137
```

(c) Comparison between two methods:

```
>> residual_gr = norm(A * x_gr - b);
>> disp(residual_gr);
    18.7707

>> residual_qr = norm(A * x_qr - b);
>> disp(residual_qr);
    18.7707

>> residual_pinv = norm(A * x_hat - b);
>> disp(residual_pinv);
    18.7707
```

Using pinv and Givens' Rotation methods lead to the same results. Yes!

Workspace	
Name ^	Value
A	8x4 double
a_q	4.3297
a_q1	16.9889
b	[13;10;-15;9;3;18;3;20]
bp	[29.4217;4.7187;-4.9134;-7.2537;-4.7...
bu	[29.4217;4.7187;-4.9134;-7.2537]
cos_theta	0.2470
G	[0.2470,0.9690;-0.9690,0.2470]
G_full	8x8 double
i	5
j	4
m	8
n	4
Q	8x8 double
R	8x4 double
residual_gr	18.7707
residual_pinv	18.7707
residual_qr	18.7707
Ru	4x4 double
sin_theta	0.9690
t	3.9238
x_gr	[0.4638;-0.1005;-0.0716;-0.4137]
x_hat	[0.4638;-0.1005;-0.0716;-0.4137]
x_qr	[0.4638;-0.1005;-0.0716;-0.4137]

2. LMS filter design

For a least mean square (LMS) adaptive filter, assume the filter is of the form finite impulse response (FIR) and 15-tap long (i.e., with 15 coefficients $b_0 \sim b_{14}$ for $x(n) \sim x(n-14)$). Given an input signal consisting of 2 frequency components

$$s(n) = \sin(2\pi n/12) + \cos(2\pi n/4)$$

develop an adaptive low pass filter design

Set the target as a low pass filter to remove the high frequency component $\cos(2\pi n/4)$ and use $\sin(2\pi n/12)$ as the desired (or training) signal for LMS adaptation. Assume the step size μ is 10^{-2} .

- write a Matlab code to simulate the LMS based adaptive filtering. Calculate the RMS (root mean square) value of the latest 16 prediction errors (i.e., $r = \sqrt{(e^2(n) + e^2(n-1) + \dots + e^2(n-15))/16}$) and the adaptation is considered being converged if this value is less than 10% of RMS (root mean square) value of the desired signal, which equals $0.1/\sqrt{2}$.

(a) LMS adaptive filter Matlab code:

```
>> L = 15;
>> N = 4000;
>> n = (0:N-1)';
>> mu_values = [10^-2, 10^-4];
>> desired_rms_target = 0.1 / sqrt(2);
>> d = sin(2*pi*n/12);
>> s = d + cos(2*pi*n/4);
>> results = struct()

>> for m_index = 1 : length(mu_values)
    mu = mu_values(m_index)
    w = zeros(L, 1);
    e = zeros(N, 1);
    r = zeros(N, 1);
    w_history = zeros(N, L);
    n_convergence = NaN;
    for i = L : N;
        u = s(i:-1:i-L+1);
        d_hat = w' * u;
        e(i) = d(i) - d_hat;
        w = w + mu * e(i) * u;
        w_history(i, :) = w';
        if i >= 16
            r(i) = sqrt(mean(e(i-15:i).^2));
            if isnan(n_convergence) && r(i) < desired_rms_target
                n_convergence = i;
            end
        end
    end
    field = sprintf('mu_%d', m_index);
    results.(field).r = r;
    results.(field).W = w_history;
    results.(field).n_convergence = n_convergence;
    results.(field).final_w = w;
end
```

L: tap-long, N: total samples,

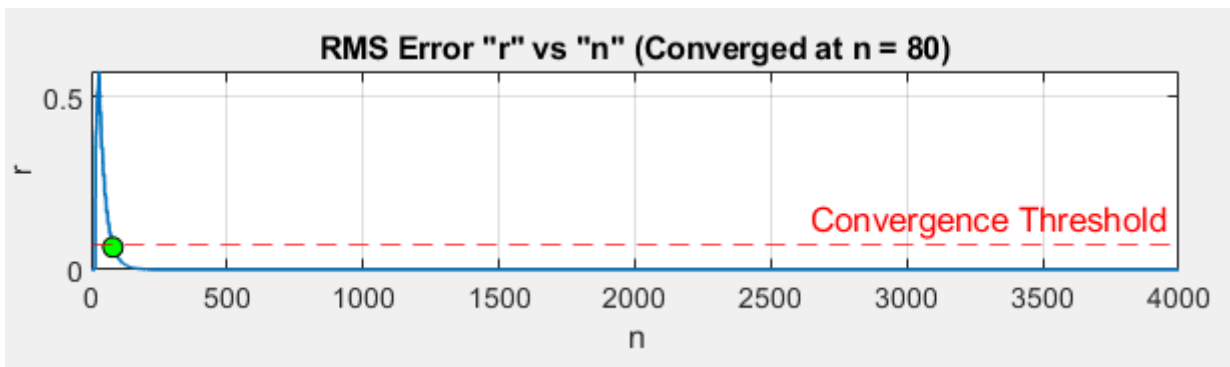
Workspace	
Name ^	Value
d	4000x1 double
d_hat	0.7524
desired_rms...	0.0707
e	4000x1 double
field	'mu_2'
i	4000
L	15
m_index	2
mu	1.0000e-04
mu1_res	1x1 struct
mu_values	[0.0100,1.0000e-04]
n	4000x1 double
N	4000
n_converge...	NaN
r	4000x1 double
results	1x1 struct
s	4000x1 double
u	15x1 double
w	15x1 double
w_history	4000x15 double

```
>> mul_res = results.mu_1;
>> fprintf('當  $\mu = 10^{-2}$  時, 收斂於  $n = \%d \backslash n$ ', results.mu_1.n_convergence);
當  $\mu = 10^{-2}$  時, 收斂於  $n = 80$ 
```

- Show the plot of “r” versus “n” and indicate when the filter converges, i.e. how many training samples are required

(b) figure Matlab code:

```
>> subplot(3,1,1);
>> plot(mul_res.r, 'LineWidth', 1); hold on;
>> yline(desired_rms_target, 'r--', 'Convergence Threshold');
>> if ~isnan(mul_res.n_convergence)
    plot(mul_res.n_convergence, mul_res.r(mul_res.n_convergence), 'ko', 'MarkerFaceColor', 'g');
    title(['RMS Error "r" vs "n" (Converged at n = ', num2str(mul_res.n_convergence), ')']);
end
>> xlabel('n'); ylabel('r'); grid on;
```

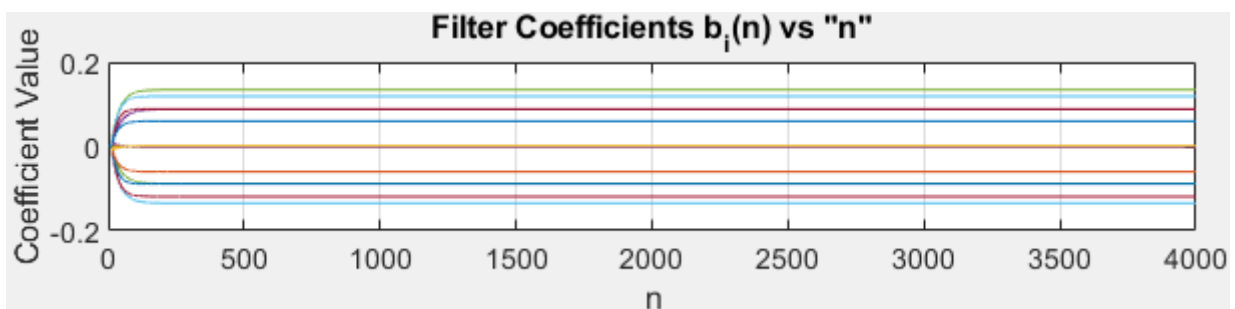


When $n = 30$, $r = 0.5746$ (max)

- Show the plot of filter coefficients $b_i(n)$, for $i = 0 \sim 14$, versus “n” and see if the values of filter coefficients remain mostly unchanged after convergence

(c) figure Matlab code:

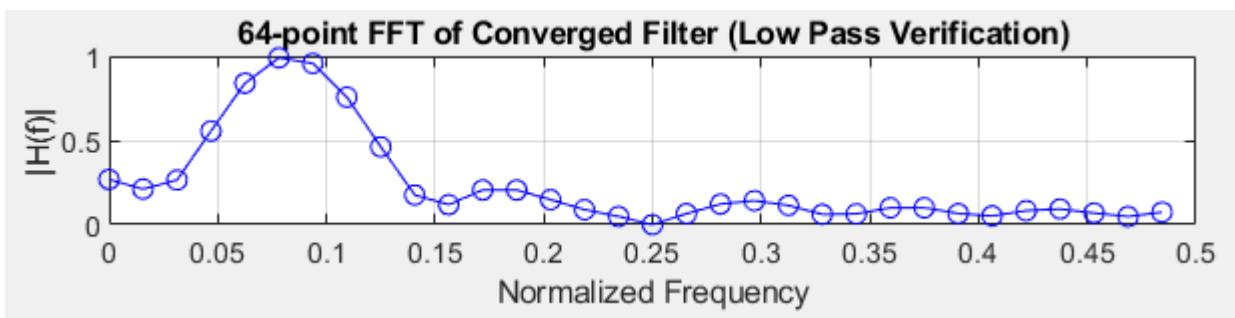
```
>> subplot(3,1,2);
>> plot(mul_res.W);
>> title('Filter Coefficients  $b_i(n)$  vs "n"');
>> xlabel('n'); ylabel('Coefficient Value'); grid on;
```



- Apply a 64-point FFT to the impulse response of the converged filter and verify the filter is indeed a low pass one. Note that the input vector to the 64-point FFT is $(b_0, b_1, \dots, b_{14}, 0, 0, \dots, 0)$ with 49 trailing zeros.

(d) figure Matlab code:

```
>> subplot(3,1,3);
>> fft_input = [mul_res.final_w; zeros(49, 1)];
>> H_freq = abs(fft(fft_input, 64));
>> plot((0:31)/64, H_freq(1:32), 'b-o');
>> title('64-point FFT of Converged Filter (Low Pass Verification)');
>> xlabel('Normalized Frequency'); ylabel('|H(f)|'); grid on;
```

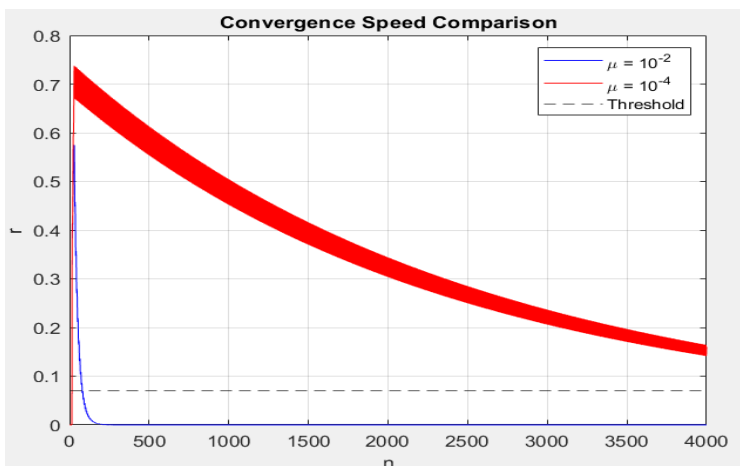


在 $f=0.08\text{Hz}$ 時 gain 最高， $f=0.25\text{Hz}$ 時衰減，說明 LMS 成功將 FIR 變為低通濾波器。

- Change the step size μ to 10^{-4} and see how the behavior of the adaptive filter changes.

(e) figure Matlab code:

```
>> figure('Name', 'Step Size Comparison for mu = 10^-4');
>> plot(results.mu_1.r, 'b'); hold on;
>> plot(results.mu_2.r, 'r');
>> yline(desired_rms_target, 'k--');
>> legend('\mu = 10^{-2}', '\mu = 10^{-4}', 'Threshold');
>> title('Convergence Speed Comparison'); xlabel('n'); ylabel('r'); grid on;
```



```
>> fprintf('當  $\mu = 10^{-4}$  時，收斂於  $n = \%d \backslash n$ ', results.mu_2.n_convergence);
```

當 $\mu = 10^{-4}$ 時，收斂於 $n = \text{NaN}$

- Conduct simulation with a sufficiently large number of samples to see how small the value of “r” can be (the convergence bias)

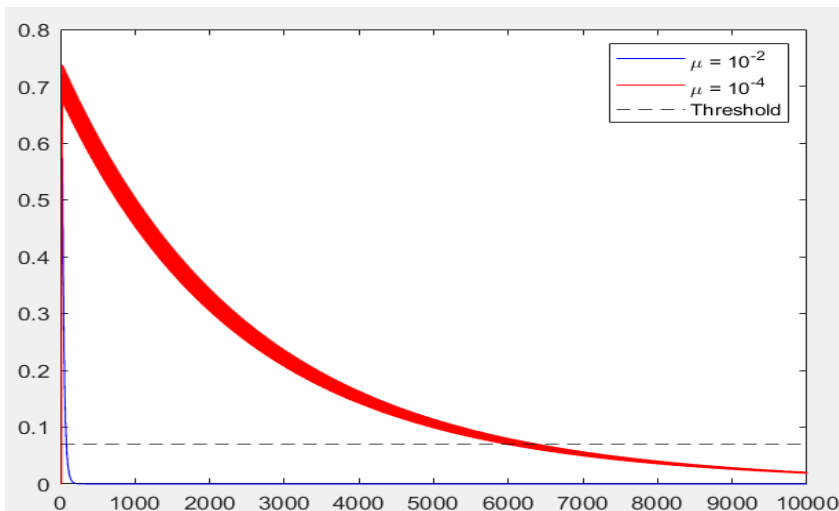
(f) figure Matlab code:

```
>> L = 15;
N = 10000;
n = (0:N-1)';
mu_values = [10^-2, 10^-4];
desired_rms_target = 0.1 / sqrt(2);
d = sin(2*pi*n/12);
s = d + cos(2*pi*n/4);
results = struct()

>> bias_mu1 = mean(results.mu_1.r(end-500:end));
>> bias_mu2 = mean(results.mu_2.r(end-500:end));
>> fprintf('Step size  $10^{-2}$  的穩態 r 值: %.6f \backslash n', bias_mu1);
Step size  $10^{-2}$  的穩態 r 值: 0.000000
>> fprintf('Step size  $10^{-4}$  的穩態 r 值: %.6f \backslash n', bias_mu2);
Step size  $10^{-4}$  的穩態 r 值: 0.021845

>> figure('Name', 'Step Size Comparison for  $\mu = 10^{-4}$  for N = 10000');
plot(results.mu_1.r, 'b'); hold on;
plot(results.mu_2.r, 'r');
yline(desired_rms_target, 'k--');
legend('\mu = 10^{-2}', '\mu = 10^{-4}', 'Threshold');
```

Workspace	
Name	Value
bias_mu1	2.4200e-13
bias_mu2	0.0218
d	10000x1 double
d_hat	0.9672
desired_rms...	0.0707
e	10000x1 double
field	'mu 2'
i	10000
L	15
m_index	2
mu	1.0000e-04
mu_values	[0.0100,1.0000e-04]
n	10000x1 double
N	10000
n_converge...	5918
r	10000x1 double
results	1x1 struct
s	10000x1 double
u	15x1 double
w	15x1 double
w_history	10000x15 double



```
>> fprintf('當  $\mu = 10^{-4}$  時，收斂於  $n = \%d \backslash n$ ', results.mu_2.n_convergence);
```

當 $\mu = 10^{-4}$ 時，收斂於 $n = 5918$

因此，當 μ 越大，其收斂樣本數會比較小，但其收斂時震盪幅度較大；反之，當 μ 越小，其收斂樣本數會比較大，但其收斂時震盪幅度較小。因此，在收斂速度與穩態偏差的 trade-off 上，較小的 μ 可以有更小的收斂偏誤，但同時也需要更長的訓練時間。

3. Discrete Wavelet Transform

For a discrete wavelet transform (DWT) adopting (9/7) filters, i.e. the low pass filter $h(i)$ is 9-taped and the high pass filter $g(i)$ is 7-taped. Both filters are linear phased and have symmetric coefficients. The filter coefficients are given in Table 1. For a corresponding inverse discrete wavelet transform, the low pass filter $q(i)$ is 7-taped and the high pass filter $p(i)$ is 9-taped. The filter coefficients are given in Table 2.

Table 1. Analysis filter coefficients for the floating point 9/7 filter

Analysis Filter Coefficients		
i	Lowpass Filter h_i	Highpass Filter g_i
0	0.852698679009	-0.788485616406
± 1	0.377402855613	0.418092273222
± 2	-0.110624404418	0.040689417609
± 3	-0.023849465020	-0.064538882629
± 4	0.037828455507	

Note: the high pass and low pass filter notations here are opposite to those in the lecture note

Table 2. Synthesis filter coefficients for the floating point 9/7 filter

Synthesis Filter Coefficients		
i	Low pass Filter q_i	High pass Filter p_i
0	0.788485616406	-0.852698679009
± 1	0.418092273222	0.377402855613
± 2	-0.040689417609	0.110624404418
± 3	-0.064538882629	-0.023849465020
± 4		-0.037828455507

- a) For a 512×512 gray scale image (will be provided along with the homework assignment), please conduct a 2-D 3-level DWT transform (as shown in Figure 1) and show the transformed result. Then conduct a 2-D 3-level IDWT to convert it back. Please compare if the reconstructed image (after IDWT) is same as the original image by calculating its PSNR value.

(a) Matlab code(HW1_3.m):

```
OGimg = imread('woman.bmp');  
[h, w, c] = size(OGimg);  
DWTimg = double(OGimg);
```

```
%% Filter Coefficients, h g are analysis, q p are synthesis  
hFilter = [0.037828455507; -0.023849465020; -0.110624404418; 0.377402855613;  
0.852698679009; 0.377402855613; -0.110624404418; -0.023849465020;  
0.037828455507];  
gFilter = [0; -0.064538882629; 0.040689417609; 0.418092273222; -0.788485616406;  
0.418092273222; 0.040689417609; -0.064538882629; 0];
```

```

qFilter = [0; -0.064538882629; -0.040689417609; 0.418092273222; 0.788485616406;
0.418092273222; -0.040689417609; -0.064538882629; 0];
pFilter = [-0.037828455507; -0.023849465020; 0.110624404418; 0.377402855613; -
0.852698679009; 0.377402855613; 0.110624404418; -0.023849465020; -
0.037828455507];

```

```

%% DWT (3-Level)

```

```

n = 3;

```

```

hArr = zeros(1, n);

```

```

wArr = zeros(1, n);

```

```

for i = 1 : n

```

```

    % Row filtering

```

```

    hFilterImg = passFilterRow(DWTimg(1:h, 1:w), hFilter);

```

```

    gFilterImg = passFilterRow(DWTimg(1:h, 1:w), gFilter);

```

```

    % Low pass: keep odd-indexed rows (1,3,5,...)

```

```

    DWTimg(1:h/2, 1:w) = hFilterImg(1:2:h, 1:w);

```

```

    % High pass: keep even-indexed rows (2,4,6,...)

```

```

    DWTimg(h/2+1:h, 1:w) = gFilterImg(2:2:h, 1:w);

```

```

    % Col filtering

```

```

    hFilterImg = passFilterCol(DWTimg(1:h, 1:w), hFilter);

```

```

    gFilterImg = passFilterCol(DWTimg(1:h, 1:w), gFilter);

```

```

    % Low pass: keep odd-indexed cols (1,3,5,...)

```

```

    DWTimg(1:h, 1:w/2) = hFilterImg(1:h, 1:2:w);

```

```

    % High pass: keep even-indexed cols (2,4,6,...)

```

```

    DWTimg(1:h, w/2+1:w) = gFilterImg(1:h, 2:2:w);

```

```

    hArr(i) = h;

```

```

    wArr(i) = w;

```

```

    w = w / 2;

```

```

    h = h / 2;

```

```

end

```

```

imwrite(uint8(DWTimg(:, :, 1)), 'DWTimage_a.bmp');

```

```

figure(1);

```

```

imshow(DWTimg(:, :, 1), []);

```

```

title('DWT Result (a)');

```

```

%% IDWT

```

```

for i = n : -1 : 1

```

```

    h = hArr(i);

```

```

    w = wArr(i);

```

```

    % Row Reconstruction

```

```

    leftBlock = DWTimg(1:h, 1:w/2);

```

```

    rightBlock = DWTimg(1:h, w/2+1:w);

```

```

    % padding insertion

```

```

    upsL = zeros(h, w);

```

```

    upsH = zeros(h, w);

```

```

    upsL(:, 1:2:w) = leftBlock;

```



```

upsH(:, 2:2:w) = rightBlock;

% filtering sum
DWTimg(1:h, 1:w) = passFilterRow(upsL, qFilter) + passFilterRow(upsH, pFilter);

% Col Reconstruction
topBlock = DWTimg(1:h/2, 1:w);
bottomBlock = DWTimg(h/2+1:h, 1:w);

% padding insertion
upsT = zeros(h, w);
upsB = zeros(h, w);
upsT(1:2:h, :) = topBlock;
upsB(2:2:h, :) = bottomBlock;

% filtering sum
DWTimg(1:h, 1:w) = passFilterCol(upsT, qFilter) + passFilterCol(upsB, pFilter);
end

imwrite(uint8(DWTimg(:, :, 1)), 'IDWTimage_a.bmp');
figure(2);
imshow(uint8(DWTimg(:, :, 1)), []);
title('IDWT Reconstructed (a)');

%% PSNR

MSE = 0;
for i = 1 : 512
    for j = 1 : 512
        MSE = MSE + (double(OGimg(i, j, 1)) - double(DWTimg(i, j, 1)))^2;
    end
end
MSE = MSE / (512^2);

MAXI = 511;
PSNR = 10 * log10((MAXI^2) / MSE);
fprintf('Part (a) PSNR = %3.4f dB\n', PSNR);

%% passFilterRow function
function passImg = passFilterRow(img, filter)
[h, w, c] = size(img);
paddingImg = padarray(img, [0 4], 'symmetric', 'both');
passImg = zeros(h, w, c);
for row = 1 : h
    for col = 5 : w + 4
        n = 1;
        for x = -4 : 4
            for channel = 1 : c
                passImg(row, col - 4, channel) = passImg(row, col - 4, channel) + ...
                    paddingImg(row, col + x, channel) * filter(n);
            end
        end
    end
end

```

```

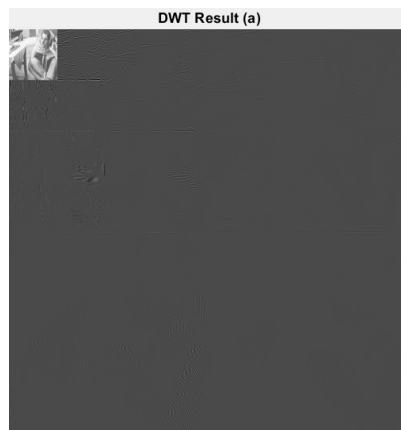
        n = n + 1;
    end
end
end
end

%% passFilterCol function
function passImg = passFilterCol(img, filter)
[h, w, c] = size(img);
paddingImg = padarray(img, [4 0], 'symmetric', 'both');
passImg = zeros(h, w, c);
for row = 5 : h + 4
    for col = 1 : w
        n = 1;
        for y = -4 : 4
            for channel = 1 : c
                passImg(row - 4, col, channel) = passImg(row - 4, col, channel) + ...
                    paddingImg(row + y, col, channel) * filter(n);
            end
            n = n + 1;
        end
    end
end
end
end

```

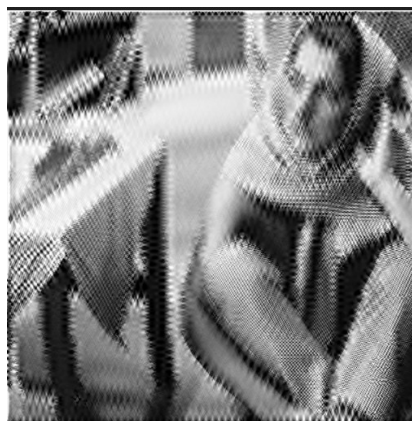
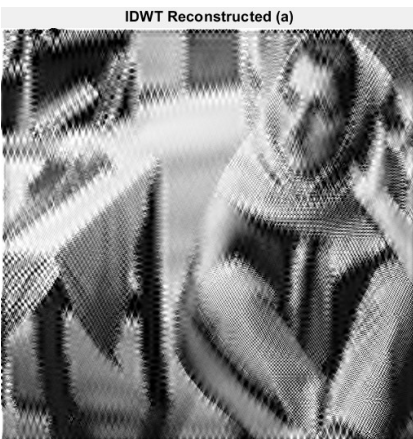
Name	Value
bottomBlock	256x512 double
c	1
DWTimg	512x512 double
gFilter	[0;-0.0645;0.0407;0.4181;-0.7885;0.4...
gFilterImg	128x128 double
h	512
hArr	[512,256,128]
hFilter	[0.0378;-0.0238;-0.1106;0.3774;0.85...
hFilterImg	128x128 double
i	512
j	512
leftBlock	512x256 double
MAXI	511
MSE	2.3742e+03
n	3
OGimg	512x512 uint8
pFilter	[-0.0378;-0.0238;0.1106;0.3774;-0.85...
PSNR	20.4132
qFilter	[0;-0.0645;-0.0407;0.4181;0.7885;0.4...
rightBlock	512x256 double
topBlock	256x512 double
upsB	512x512 double
upsH	512x512 double
upsL	512x512 double
upsT	512x512 double
w	512
wArr	[512,256,128]

(1) Original Image(512X512): (2) DWT image figure & DWTimage_a.bmp:



(3) IDWT image figure & IDWTimage_a.bmp:

512 x 512 257.1 KB



```
>> HW1_3
```

```
Part (a) PSNR = 20.4132 dB
```

- b)** By setting all three level 1 sub-bands HL1, LH1 and HH1 coefficients to zeros and perform IDWT. See how the reconstructed image is different from the original one and calculate its PSNR value.

(b) Matlab code(HW1_3_b.m):

```
OGimg = imread('woman.bmp');
[h, w, c] = size(OGimg);
DWTimg = double(OGimg);

%% Filter Coefficients, h g are analysis, q p are synthesis
hFilter = [0.037828455507; -0.023849465020; -0.110624404418; 0.377402855613;
0.852698679009; 0.377402855613; -0.110624404418; -0.023849465020;
0.037828455507];
gFilter = [0; -0.064538882629; 0.040689417609; 0.418092273222; -0.788485616406;
0.418092273222; 0.040689417609; -0.064538882629; 0];
qFilter = [0; -0.064538882629; -0.040689417609; 0.418092273222; 0.788485616406;
0.418092273222; -0.040689417609; -0.064538882629; 0];
pFilter = [-0.037828455507; -0.023849465020; 0.110624404418; 0.377402855613; -
0.852698679009; 0.377402855613; 0.110624404418; -0.023849465020; -
0.037828455507];

%% DWT (3-Level)
n = 3;
hArr = zeros(1, n);
wArr = zeros(1, n);

for i = 1 : n
    % Row filtering
    hFilterImg = passFilterRow(DWTimg(1:h, 1:w), hFilter);
    gFilterImg = passFilterRow(DWTimg(1:h, 1:w), gFilter);
    % Low pass: keep odd-indexed rows (1,3,5,...)
    DWTimg(1:h/2, 1:w) = hFilterImg(1:2:h, 1:w);
    % High pass: keep even-indexed rows (2,4,6,...)
    DWTimg(h/2+1:h, 1:w) = gFilterImg(2:2:h, 1:w);

    % Col filtering
    hFilterImg = passFilterCol(DWTimg(1:h, 1:w), hFilter);
    gFilterImg = passFilterCol(DWTimg(1:h, 1:w), gFilter);
    % Low pass: keep odd-indexed cols (1,3,5,...)
    DWTimg(1:h, 1:w/2) = hFilterImg(1:h, 1:2:w);
    % High pass: keep even-indexed cols (2,4,6,...)
    DWTimg(1:h, w/2+1:w) = gFilterImg(1:h, 2:2:w);

    hArr(i) = h;
    wArr(i) = w;
    w = w / 2;
end
```

```

    h = h / 2;
end

imwrite(uint8(DWTimg(:, :, 1)), 'DWTimage_b.bmp');
figure(1);
imshow(DWTimg(:, :, 1), []);
title('DWT Result (b)');

%% IDWT
h1 = hArr(1);
w1 = wArr(1);
DWTimg(1:h1/2, w1/2+1:w1) = 0;    % HL1
DWTimg(h1/2+1:h1, 1:w1/2) = 0;    % LH1
DWTimg(h1/2+1:h1, w1/2+1:w1) = 0; % HH1
for i = n : -1 : 1
    h = hArr(i);
    w = wArr(i);

    % Row Reconstruction
    topBlock = DWTimg(1:h/2, 1:w);
    bottomBlock = DWTimg(h/2+1:h, 1:w);

    upsTop = zeros(h, w);
    upsTop(1:2:h, :) = topBlock;
    upsBottom = zeros(h, w);
    upsBottom(2:2:h, :) = bottomBlock;

    DWTimg(1:h, 1:w) = passFilterRow(upsTop, qFilter) + passFilterRow(upsBottom,
pFilter);

    % Col Reconstruction
    leftBlock = DWTimg(1:h, 1:w/2);
    rightBlock = DWTimg(1:h, w/2+1:w);

    upsLeft = zeros(h, w);
    upsLeft(:, 1:2:w) = leftBlock;
    upsRight = zeros(h, w);
    upsRight(:, 2:2:w) = rightBlock;

    DWTimg(1:h, 1:w) = passFilterCol(upsLeft, qFilter) + passFilterCol(upsRight, pFilter);
end

imwrite(uint8(DWTimg(:, :, 1)), 'IDWTimage_b.bmp');
figure(2);
imshow(uint8(DWTimg(:, :, 1)), []);
title('IDWT Reconstructed (b)');

%% PSNR

MSE = 0;
for i = 1 : 512

```

```

    for j = 1 : 512
        MSE = MSE + (double(OGimg(i, j, 1)) - double(DWTimg(i, j, 1)))^2;
    end
end
MSE = MSE / (512^2);

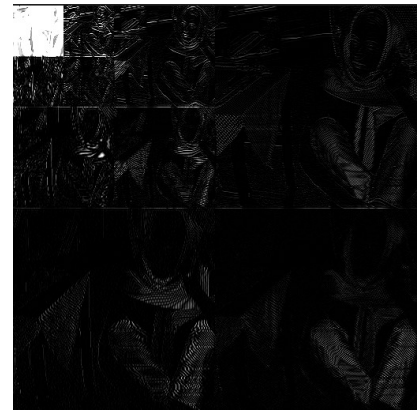
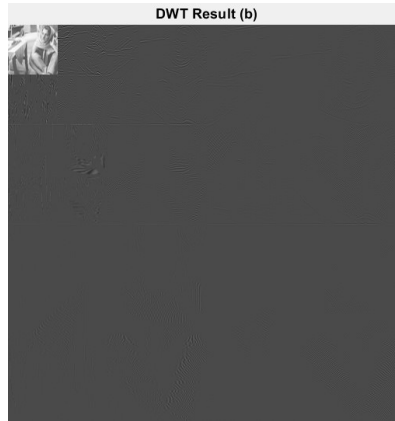
MAXI = 511;
PSNR = 10 * log10((MAXI^2) / MSE);
fprintf('Part (b) PSNR = %3.4f dB\n', PSNR);

%% passFilterRow function
function passImg = passFilterRow(img, filter)
    [h, w, c] = size(img);
    paddingImg = padarray(img, [0 4], 'symmetric', 'both');
    passImg = zeros(h, w, c);
    for row = 1 : h
        for col = 5 : w + 4
            n = 1;
            for x = -4 : 4
                for channel = 1 : c
                    passImg(row, col - 4, channel) = passImg(row, col - 4, channel) + ...
                        paddingImg(row, col + x, channel) * filter(n);
                end
                n = n + 1;
            end
        end
    end
end

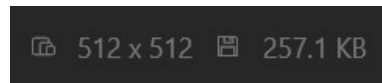
%% passFilterCol function
function passImg = passFilterCol(img, filter)
    [h, w, c] = size(img);
    paddingImg = padarray(img, [4 0], 'symmetric', 'both');
    passImg = zeros(h, w, c);
    for row = 5 : h + 4
        for col = 1 : w
            n = 1;
            for y = -4 : 4
                for channel = 1 : c
                    passImg(row - 4, col, channel) = passImg(row - 4, col, channel) + ...
                        paddingImg(row + y, col, channel) * filter(n);
                end
                n = n + 1;
            end
        end
    end
end

```

(1) Original Image(512X512): (2) DWT image figure & DWTimage_b.bmp:



(3) IDWT image figure & IDWTimage_b.bmp:



```
>> HW1_3_b  
Part (b) PSNR = 10.4704 dB
```

將 LH1、HL1、HH1 進行清 0 動作，意指將影像高頻部分濾除，達到低通濾波的效果，但造成了 PSNR 的下降，影像的重建程度降低，影像更顯模糊。

<ps> 2. LMS filter design command window code

```
>> L = 15;  
N = 4000;  
n = (0:N-1)';  
mu_values = [10^-2, 10^-4];  
desired_rms_target = 0.1 / sqrt(2);  
d = sin(2*pi*n/12);  
s = d + cos(2*pi*n/4);
```

```
results = struct()
```

results = struct with no fields.

```
>> for m_index = 1 : length(mu_values)
    mu = mu_values(m_index)
    w = zeros(L, 1);
    e = zeros(N, 1);
    r = zeros(N, 1);
    w_history = zeros(N, L);
    n_convergence = NaN;
    for i = L : N;
        u = s(i:-1:i-L+1);
        d_hat = w' * u;
        e(i) = d(i) - d_hat;
        w = w + mu * e(i) * u;
        w_history(i, :) = w';
        if i >= 16
            r(i) = sqrt(mean(e(i-15:i).^2));
            if isnan(n_convergence) && r(i) < desired_rms_target
                n_convergence = i;
            end
        end
    end
end
field = sprintf('mu_%d', m_index);
results.(field).r = r;
results.(field).W = w_history;
results.(field).n_convergence = n_convergence;
results.(field).final_w = w;
end
```

```
mu = 0.0100
```

```
mu = 1.0000e-04
```

```
>> mu1_res = results.mu_1;
```



```

fprintf('當 mu = 10^-2 時，收斂於 n = %d\n', results.mu_1.n_convergence);
當 mu = 10^-2 時，收斂於 n = 80
>> subplot(3,1,1);
>> plot(mu1_res.r, 'LineWidth', 1); hold on;
>> yline(desired_rms_target, 'r--', 'Convergence Threshold');
>> if ~isnan(mu1_res.n_convergence)
    plot(mu1_res.n_convergence, mu1_res.r(mu1_res.n_convergence), 'ko',
'MarkerFaceColor', 'g');
    title(['RMS Error "r" vs "n" (Converged at n = ', num2str(mu1_res.n_convergence), ')']);
end
>> xlabel('n'); ylabel('r'); grid on;
>> subplot(3,1,2);
plot(mu1_res.W);
title('Filter Coefficients b_i(n) vs "n"');
xlabel('n'); ylabel('Coefficient Value'); grid on;
>> subplot(3,1,3);
>> fft_input = [mu1_res.final_w; zeros(49, 1)];
>> H_freq = abs(fft(fft_input, 64));
>> plot((0:31)/64, H_freq(1:32), 'b-o');
>> title('64-point FFT of Converged Filter (Low Pass Verification)');
>> xlabel('Normalized Frequency'); ylabel('|H(f)|'); grid on;
>> figure('Name', 'Step Size Comparison for mu = 10^-4');
>> plot(results.mu_1.r, 'b'); hold on;
>> plot(results.mu_2.r, 'r');
>> yline(desired_rms_target, 'k--');
>> legend('\mu = 10^{-2}', '\mu = 10^{-4}', 'Threshold');
>> title('Convergence Speed Comparison'); xlabel('n'); ylabel('r'); grid on;
>> fprintf('當 mu = 10^-4 時，收斂於 n = %d\n', results.mu_2.n_convergence);
當 mu = 10^-4 時，收斂於 n = NaN

```